



常量与运算符



什么是常量?



常量 是在程序运行过程中**其值不能改变**的量。

- 使用 *const* 关键字定义
- 定义时必须初始化
- 之后任何试图修改的操作都会导致编译错误

💡 **生活类比**: 常量就像是一个**封死的盒子**, 里面的东西一旦放进去就不能再换了。

```
const double PI = 3.14159;    // 定义圆周率常量
const int MAX_SIZE = 100;     // 定义最大尺寸常量
```

为什么使用常量？



优点	说明
提高可读性	用有意义的名称代替“魔法数字”
便于维护	只需修改一处，所有引用自动更新
防止误改	编译器会检查，避免意外修改

示例对比：

```
// ✗ 不好的写法（魔法数字）  
double area = 3.14159 * r * r;
```

```
// ✓ 好的写法（使用常量）  
const double PI = 3.14159;  
double area = PI * r * r;
```

算术运算符



运算符	含义	示例	结果
+	加法	<code>10 + 3</code>	13
-	减法	<code>10 - 3</code>	7
*	乘法	<code>10 * 3</code>	30
/	除法	<code>10 / 3</code>	3（整数除法）
	取余（模运算）	<code>10 % 3</code>	1

⚠ 注意：

- 整数除法会**截断**小数部分（向零取整）
- 只能用于整数操作数



整数除法与浮点数除法



整数 ÷ 整数 = 整数 (截断小数)

```
int a = 10, b = 3;  
cout << a / b;    // 输出 3, 而不是 3.333...
```

浮点数 ÷ 整数 = 浮点数 (自动类型提升)

```
double a = 10.0;  
int b = 3;  
cout << a / b;    // 输出 3.333...
```

强制转换为浮点数

```
int a = 10, b = 3;  
cout << (double)a / b;    // 输出 3.333...
```

赋值运算符



运算符	含义	示例	等价于
=	直接赋值	<code>a = 5;</code>	<code>a = 5;</code>
+ =	加后赋值	<code>a += 3;</code>	<code>a = a + 3;</code>
- =	减后赋值	<code>a -= 3;</code>	<code>a = a - 3;</code>
* =	乘后赋值	<code>a *= 3;</code>	<code>a = a * 3;</code>
/ =	除后赋值	<code>a /= 3;</code>	<code>a = a / 3;</code>
	取余后赋值	<code>a %= 3;</code>	<code>a = a % 3;</code>

示例：

```
int a = 10;  
a += 5;    // a 变为 15  
a *= 2;    // a 变为 30
```

类型转换（隐式转换）



隐式转换：编译器自动将一种类型转换为另一种类型。

- **提升：**小类型 → 大类型（安全，不丢失信息）
- **降级：**大类型 → 小类型（可能丢失信息，编译器会警告）

```
int a = 10;  
double b = 3.14;  
double c = a + b;    // a 自动提升为 double, 结果 13.14  
int d = a + b;       // 警告! double 转为 int, 结果 13 (截断)
```

常见的隐式转换方向：

*bool char short int unsigned int long
unsigned long long float double long double*

类型转换（显式转换）



显式转换（强制类型转换）：程序员手动指定转换方式。

```
int a = 10, b = 3;  
double result = (double)a / b;    // C 风格强制转换
```

C++ 也提供了更安全的转换方式（后续课程会细讲），目前我们常用 C 风格。

应用场景：

- 需要精确的浮点除法结果
- 将字符转为 ASCII 码值
- 将数值转为字符（如 `char(65)` 得到 'A'）

```
char c = 'A';  
int ascii = (int)c;    // ascii = 65  
cout << char(ascii + 1); // 输出 'B'
```

实战①：计算圆的面积



题目描述

输入圆的半径 r （实数），输出圆的面积，保留两位小数。

- 输入：一个实数 r ($0 < r \leq 10^4$)
- 输出：圆的面积，保留两位小数
- 公式： $S = \pi \times r^2$ ，取 $\pi = 3.14159$



实战①：计算圆的面积



解题思路

1. 定义常量 $PI = 3.14159$
2. 声明 *double* 变量 r 和 $area$
3. 读取半径、计算面积: $area = PI * r * r$
4. 输出结果, 保留两位小数

参考代码

```
const double PI = 3.14159;  
double r, area;  
cin >> r;  
area = PI * r * r;  
cout << fixed << setprecision(2) << area;
```

fixed 和 *setprecision(2)* 用于控制小数位数 (需要 `<iomanip>`, 但 `<bits/stdc++.h>` 已包含)



实战②：求两个整数的商和余数



题目描述

输入两个正整数 a 和 b ($b \neq 0$)，输出 $a \div b$ 的**商**和**余数**，用空格分隔。

- 输入：两个整数，如 `17 5`
- 输出：商和余数，如 `3 2`

解题思路

- 商： `a / b`（整数除法）
- 余数： `a % b`

实战②：参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << a / b << " " << a % b;
    return 0;
}
```

💡 思考：如果 b 为负数，余数的符号会如何？C++ 中余数的符号与被除数相同。

实战③：温度转换（华氏转摄氏）



题目描述

输入华氏温度 F （实数），输出对应的摄氏温度 C ，保留一位小数。

- 转换公式： $C = \frac{5}{9} \times (F - 32)$
- 输入样例：100
- 输出样例：37.8

实战③：参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    double F, C;
    cin >> F;
    C = 5.0 / 9 * (F - 32);    // 注意 5.0 而不是 5，否则整数除法
    cout << fixed << setprecision(1) << C;
    return 0;
}
```

⚠ 易错点：5/9 结果是 0，必须写成 `5.0 / 9` 或 `(double)5 / 9`

实战④：交换两个变量的值（升级版）



题目描述

输入两个整数 a 和 b ，交换它们的值后输出。要求不使用第三个变量。

- 输入样例：3 7
- 输出样例：7 3

解法：使用加法和减法

```
int a, b;  
cin >> a >> b;  
a = a + b;  
b = a - b;  
a = a - b;  
cout << a << " " << b;
```

也可以使用异或（`^`）运算符，但加法更直观。



复合赋值运算符实战



题目描述

输入一个整数 n ，依次执行以下操作：

1. 加 5 $\rightarrow$ 2. 乘以 2
2. 减去 10 $\rightarrow$ 除以 3
3. 输出最终结果。

参考代码

```
int n;  
cin >> n;  
n += 5;           // n = n + 5  
n *= 2;           // n = n * 2  
n -= 10;          // n = n - 10  
n /= 3;           // n = n / 3 (整数除法)  
cout << n;
```

输入 10 $\rightarrow$ 15 $\rightarrow$ 30 $\rightarrow$ 20 $\rightarrow$ 6 (输出 6)

数据类型选择与运算符注意事项



情况	建议
需要精确小数结果	使用 <code>double</code> 并确保至少一个操作数为浮点数
只关心整数商和余数	使用 <code>int</code> ，用 <code>/</code> 和 <code>%</code>
定义不会改变的值	使用 <code>const</code> 常量
避免整数溢出	注意 <code>int</code> 范围（约 ± 21 亿），大数用 <code>long long</code>

溢出示例：

```
int a = 2000000000;  
int b = 2000000000;  
int c = a + b;    // 溢出！结果不可预测
```

本节回顾



- ✓ 常量 —— `const` 定义，值不可变
- ✓ 算术运算符 —— `+` `-` `*` `/` `%`，注意整数除法
- ✓ 赋值运算符 —— `=` `+=` `-=` `*=` `/=` `%=`
- ✓ 类型转换 —— 隐式（自动）和显式（强制）
- ✓ 实战技巧 —— 浮点数除法、格式化输出、防止溢出

下节课预告



输入输出进阶与格式化

- `cin` / `cout` 的格式化控制
- `scanf` / `printf` (C 风格)
- 操纵器: `setw`、`setfill`、`left` / `right`
- 练习题: 计算多项式的值

“不积跬步，无以至千里。运算符是代码的动词，让数据流动起来。”

谢谢大家！



掌握常量与运算符，让程序更稳健、更高效！

Q & A